1. Be familiar with pigeonhole-sort algorithm, counting-sort algorithm, radix-sort algorithm, and their time complexity analysis. Also, can illustrate the radix sort algorithm by using examples [See theory HW2.2, Questions 1a and 2].
2. Understand the definition of stable sorting, and why stable sorting algorithm is important for radix sort. [See ch6-sorting-heap-linear, slides 44-45 and 49-51, and theory HW2.2, Question 1b].
3. Can apply dynamic programing to solve problem; Know that the general steps for how to use the dynamic programing (principal of optimality, deriving a recursive formula, figuring out an order to fill the table, constructing the solution). [See ch7- dynamic-programming, slide 74-85; Theory HW3.1, question 1].
4. Be familiar with Floyd's algorithm, and can find all pairs shortest paths for a given graph by constructing the matrix D, which contains the lengths of shortest paths, and the matrix P, which contains the indices of the intermediate vertices on the shortest paths [See ch7-dynamic-programming, slide 37-41; Theory HW3.1, question 2].
5. Understand the difference between adjacency matrix and adjacency list for a graph representation, their advantage and disadvantage [See ch8-graphs-basics, slide 16-23].
6. Understand Breath First Search (BFS) and Depth First Search (DFS) algorithm; can label the first visit time and exit time of each node for a given graph; know how to generate a DFS tree for a given graph; understand when you will have a back edge [See ch8-graphs-basics, slide 33-41; Theory HW3.2, question 1].
7. Be familiar with Prim's algorithm and Kruskal's algorithm, and can generate a minimum spanning tree (MST) using both algorithms for a given graph [See ch9-greedy-prim, slide 16-19, and ch10-greedy-disjointsets-kruskal, slide 23-30; Theory HW3.2, questions 4 and 5].
8. Can apply greedy approach/algorithm to solve problem; Know that the greedy strategy does not guarantee an optimum solution, and a proof is needed if you want to show that the greedy strategy provides an optimum solution [See ch9-greedy-prim, slide 2-10 and ch12-knapsck-greedy vs. dynamic programming, slide 11-15; Theory HW3.2, question 3].
9. Be familiar with Dijkstra's algorithm and understand how the Dijkstra's algorithm selects the nodes at each step [See ch11-greedy-dijkstra, slide 6-13].
10. Understand the difference between 0/1 Knapsack problem and Fractional Knapsack problem, and how to calculate the optimal benefit for Fractional Knapsack problem [See ch12-knapsack-greedy vs. dynamic programming, slide 4, 5 and slide 23,24].
11. Can calculate the optimum benefit for 0/1 knapsack problem using dynamic programming approach for a given example and be familiar with its time complexity [See ch12-knapsack-greedy vs. dynamic programming, slide 30-37; Theory HW4, question 4]
12. Understand why backtracking algorithm can be more efficient compared to complete state space tree for both sum of subset problem and 0/1 knapsack problem; Know when a state space tree node is non-promising for both sum of subset problem and 0/1 knapsack problem [See ch13-backtracking, slide 9-16 and slide 25-32; Theory HW4, questions 1 and 2 and 3].
13. Understand the purpose of amortized analysis and can use aggregate analysis to find the amortized cost for the sequence of n operations for different examples [See Lecture notes Ch15, slides 3, 5-12; Theory HW4, question 5].

14. Understand how the accounting method works and can use accounting method to find the amortized cost for the sequence of n operations for different examples [See Lecture notes Ch15, slides 3, 5-12, 13-16].
15. Know the relationship between NP and P ($P \subset NP$) and the condition when NP = P; Understand that nobody knows whether P = NP or P <> NP. Specifically, if a problem (e.g. sorting algorithm) belongs to P, it will belong to NP (because $P \subset NP$) [See ch16-np, slide 24, 42, 51; Theory HW4, question 6].
16. Understand the definition of P and NP, which can be verified in polynomial time [See ch16-np, slide 13, 34, 41, 42]. Understand the relationship between NP-complete and NP-hard [See ch16-np, slide 50]. Can give examples of problems belonging to P, NP, NP-complete, and NP-hard. Specifically, 0/1 knapsack problem, 3SAT, traveling salesman problem are all NP-complete problems, NP problems, and also NP-hard problems, but nobody knows whether they belong to P or not.